1 Цель работы

Изучить цепочечные команды и приобрести практические навыки их применения для обработки строк символов.

2 Постановка задачи

Написать программу на языке ассемблера, которая по исходной строке формирует палиндром, используя цепочечные команды.

Способ построения: к исходной строке S дописывается её обратная запись S^R. Результат $P = S\,S^R$ одинаково читается в обоих направлениях. Например, `abcd` преобразуется в `abcddcba`.

Программа предназначена для 32-разрядной Windows и собирается MASM. Вводится строка длиной до 255 символов в однобайтовой кодировке CP866, включая русские буквы. Пробелы, регистр и знаки препинания сохраняются. Пустая строка даёт пустой результат; слишком длинный ввод отклоняется.

3 Алгоритм

1. Сбросить DF командой `CLD` в начале `MAIN`. Получить дескрипторы консоли и прочитать исходную строку функцией `ReadConsoleA`. Проверить длину и удалить из учитываемой длины завершающие символы CR и LF. Обозначить длину строки через n.

2. Сбросить флаг направления командой `CLD`. Поместить адреса исходного и результирующего буферов в `ESI` и `EDI`, а n в `ECX`. Скопировать строку командой `REP MOVSB`.

3. Если $n = 0$, перейти к выводу. Иначе установить `ESI` на последний символ исходной строки и выполнить n повторений: `STD; LODSB; CLD; STOSB`. `LODSB` читает символ в `AL` и уменьшает `ESI`, а `STOSB` записывает символ и увеличивает `EDI`.

4. Добавить CR и LF в конец результата. Вывести $2n + 2$ байта функцией `WriteConsoleA` и завершить программу.

4 Используемые команды и функции

Для сборки использованы MASM (`ml.exe`) и Microsoft Link (`link.exe`) из Visual Studio 2026, инструменты x86.

- `REP MOVSB` — копирование n байт из `[ESI]` в `[EDI]`, счётчик повторений — `ECX`.
- `STD`, `CLD` — выбор направления движения указателей; `LODSB`, `STOSB` — чтение байта в `AL` и запись байта из `AL`.
- `GetStdHandle`, `ReadConsoleA`, `WriteConsoleA` — получение дескрипторов, ввод и вывод; `ExitProcess` — завершение.
- `SetConsoleCP`, `SetConsoleOutputCP` — установка CP866; `PrintText` — вывод заданного количества байтов.

5 Тестирование

После сборки программа проверена скриптом `tests/run_tests.py` на 15 строках из таблицы. Во всех случаях получены ожидаемые результат и код завершения. Для допустимого ввода также проверены длина $2n$ и симметричность результата. Журнал сборки сохранён в `tests/build.log`, полные протоколы — в `code/tests.txt` и `tests/results.json`.

В таблице ε обозначает пустую строку, $\sqcup$ — пробел, а степень — число повторений строки.

№	Исходная строка	Фактический результат	Код выхода
1	ε	ε	0
2	`a`	`aa`	0
3	`ab`	`abba`	0
4	`abc`	`abccba`	0
5	`abcd`	`abcddcba`	0
6	`aba`	`abaaba`	0
7	`aaaa`	`aaaaaaaa`	0
8	`a b`	`a bb a`	0
9	$\sqcup a\sqcup$	$\sqcup a \sqcup \sqcup a\sqcup$	0
10	`Ab1!`	`Ab1!!1bA`	0
11	`тест`	`тесттсет`	0
12	`а роза`	`а розаазор а`	0
13	$(\texttt{abc})^{85}$, 255 символов	$(\texttt{abc})^{85}(\texttt{cba})^{85}$, 510 символов	0
14	$\texttt{a}^{256}$	Ошибка: превышена длина ввода	1
15	$\texttt{b}^{300}$	Ошибка: превышена длина ввода	1

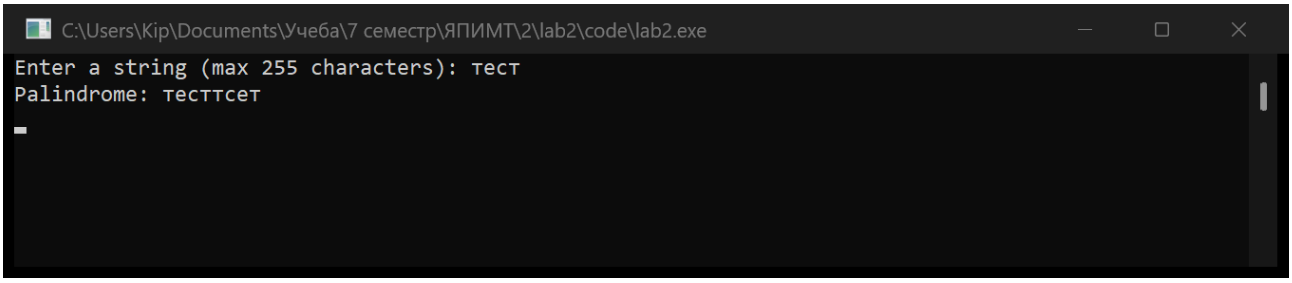
Скриншоты консоли Windows

Ниже приведены снимки окон консоли при выполнении двух тестов.

Тест № 11

Назначение: обработка русских букв в CP866.

Ожидается: `тесттсет`, код завершения 0.

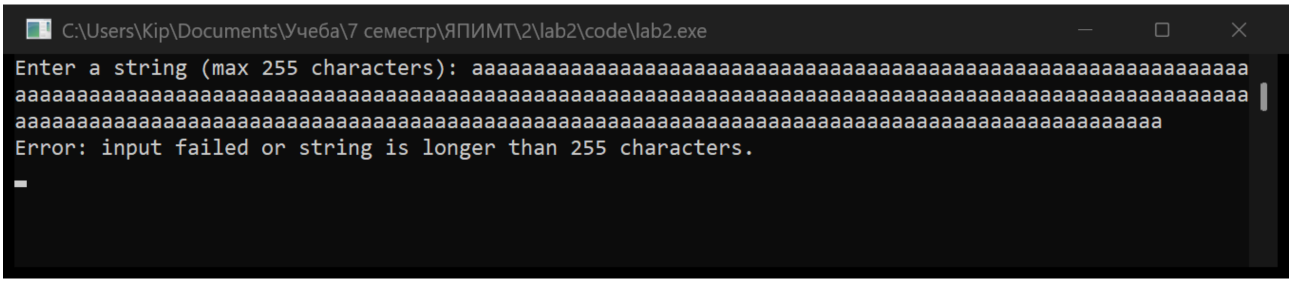

C:\Users\Kip\Documents\Учеба\7 семестр\ЯПИМТ\2\lab2\code\lab2.exe

```
Enter a string (max 255 characters): тест
Palindrome: тесттсет
```

Тест № 14

Назначение: отклонение строки из 256 символов `a`.

Ожидается: сообщение об ошибке, код завершения 1.


C:\Users\Kip\Documents\Учеба\7 семестр\ЯПИМТ\2\lab2\code\lab2.exe

```
Enter a string (max 255 characters): aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Error: input failed or string is longer than 255 characters.
```

В комплект работы входят исходник `code/main.asm`, исполняемый файл, журнал сборки, скрипт и результаты проверок. Для повторной проверки после сборки из корня репозитория выполняется команда:

```
py -3 tests/run_tests.py --screenshots
```

Контрольные суммы исходника и EXE сохранены в `tests/run-info.json`.

6 Код программы

```
.386
.MODEL FLAT, STDCALL
.STACK 4096

EXTERN GetStdHandle@4:PROC
EXTERN SetConsoleCP@4:PROC
EXTERN SetConsoleOutputCP@4:PROC
EXTERN ReadConsoleA@20:PROC
EXTERN WriteConsoleA@20:PROC
EXTERN ExitProcess@4:PROC

MAX_LENGTH EQU 255

.DATA
prompt       DB 'Enter a string (max 255 characters): '
promptLength EQU $ - prompt
resultText   DB 'Palindrome: '
```

```
resultLength EQU $ - resultText
errorText    DB 'Error: input failed or string is longer than 255 characters.',13,10
errorLength  EQU $ - errorText
inputHandle  DD ?
outputHandle DD ?
readCount    DD ?
written      DD ?
stringLength DD ?
inputBuffer  DB MAX_LENGTH + 3 DUP (?)
outputBuffer DB 2 * MAX_LENGTH + 2 DUP (?)

.CODE
; Вывод EDX байтов из буфера с адресом EAX.
PrintText PROC
    PUSH 0
    PUSH OFFSET written
    PUSH EDX
    PUSH EAX
    PUSH outputHandle
    CALL WriteConsoleA@20
    RET
PrintText ENDP

MAIN PROC
    CLD
    ; В CP866 русская буква занимает один байт.
    PUSH 866
    CALL SetConsoleCP@4
    PUSH 866
    CALL SetConsoleOutputCP@4

    PUSH -10
    CALL GetStdHandle@4
    MOV inputHandle, EAX
    PUSH -11
    CALL GetStdHandle@4
    MOV outputHandle, EAX

    MOV EAX, OFFSET prompt
    MOV EDX, promptLength
    CALL PrintText

    PUSH 0
    PUSH OFFSET readCount
    PUSH SIZEOF inputBuffer
    PUSH OFFSET inputBuffer
    PUSH inputHandle
    CALL ReadConsoleA@20
    TEST EAX, EAX
    JZ inputError

    ; Консоль добавляет CR и LF; они не входят в исходную строку.
    MOV ECX, readCount
    CMP ECX, 2
    JB inputError
    SUB ECX, 2
    CMP ECX, MAX_LENGTH
    JA inputError
    CMP BYTE PTR inputBuffer[ECX], 13
    JNE inputError
    CMP BYTE PTR inputBuffer[ECX+1], 10
    JNE inputError
    MOV stringLength, ECX

    ; Копирование исходной строки.
    CLD
    MOV ESI, OFFSET inputBuffer
    MOV EDI, OFFSET outputBuffer
    REP MOVSB

    ; Дописывание символов в обратном порядке.
    MOV ECX, stringLength
    JECXZ palindromeReady
    LEA ESI, inputBuffer[ECX-1]
copyBackwards:
    STD
    LODSB
    CLD
    STOSB
    LOOP copyBackwards

palindromeReady:
    ; Добавление перевода строки.
    MOV AL, 13
    STOSB
    MOV AL, 10
    STOSB

    MOV EAX, OFFSET resultText
    MOV EDX, resultLength
    CALL PrintText

    MOV EAX, OFFSET outputBuffer
    MOV EDX, stringLength
    ADD EDX, EDX
    ADD EDX, 2
    CALL PrintText

    PUSH 0
    CALL ExitProcess@4

inputError:
    MOV EAX, OFFSET errorText
    MOV EDX, errorLength
    CALL PrintText
    PUSH 1
    CALL ExitProcess@4
MAIN ENDP
END MAIN
```

7 Вывод

В ходе работы я использовал цепочечные команды для формирования палиндрома. Сначала исходная строка копируется в результирующий буфер командой `REP MOVSB`, затем её символы дописываются в обратном порядке с помощью `LODSB` и `STOSB`. Направление чтения и записи задаётся командами `STD` и `CLD`. Отдельно обработаны пустая строка и ограничение ввода до 255 символов. При проверке 15 случаев получены ожидаемые результаты, включая русский ввод и отклонение слишком длинной строки.